Grundlagen der Informationsvisualisierung

Exercise Session:
Week 1

Madhav Poddar,
Leah Mühlöder
Prof. Dr. Fabian Beck

Summer Semester
2026

University of Bamberg

Formal aspects & Questions

Data Visualization Examples

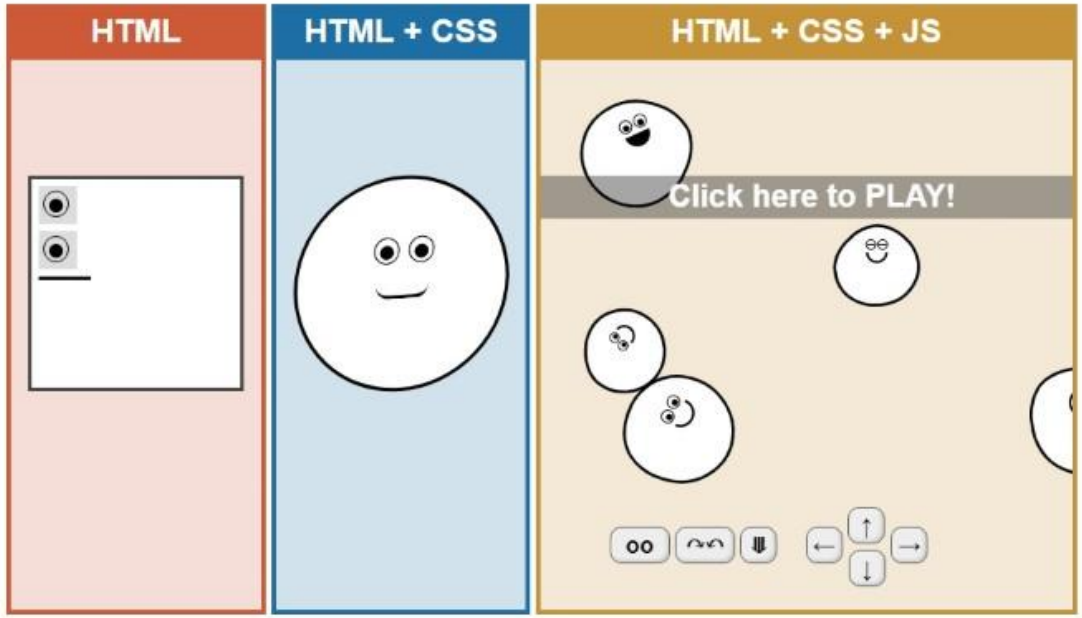
- Besides the ones we saw in lecture this week, where else have you spotted data visualizations?
  - Take 5 minutes to find one you've seen before—or discover a cool new example!

- Now, introduce yourself (name, study program) and tell us a bit about the visualization you found!

Web Technologies

Web Technologies

https://medium.com/@gargatulya14/web-technology-9a34836bd2bd

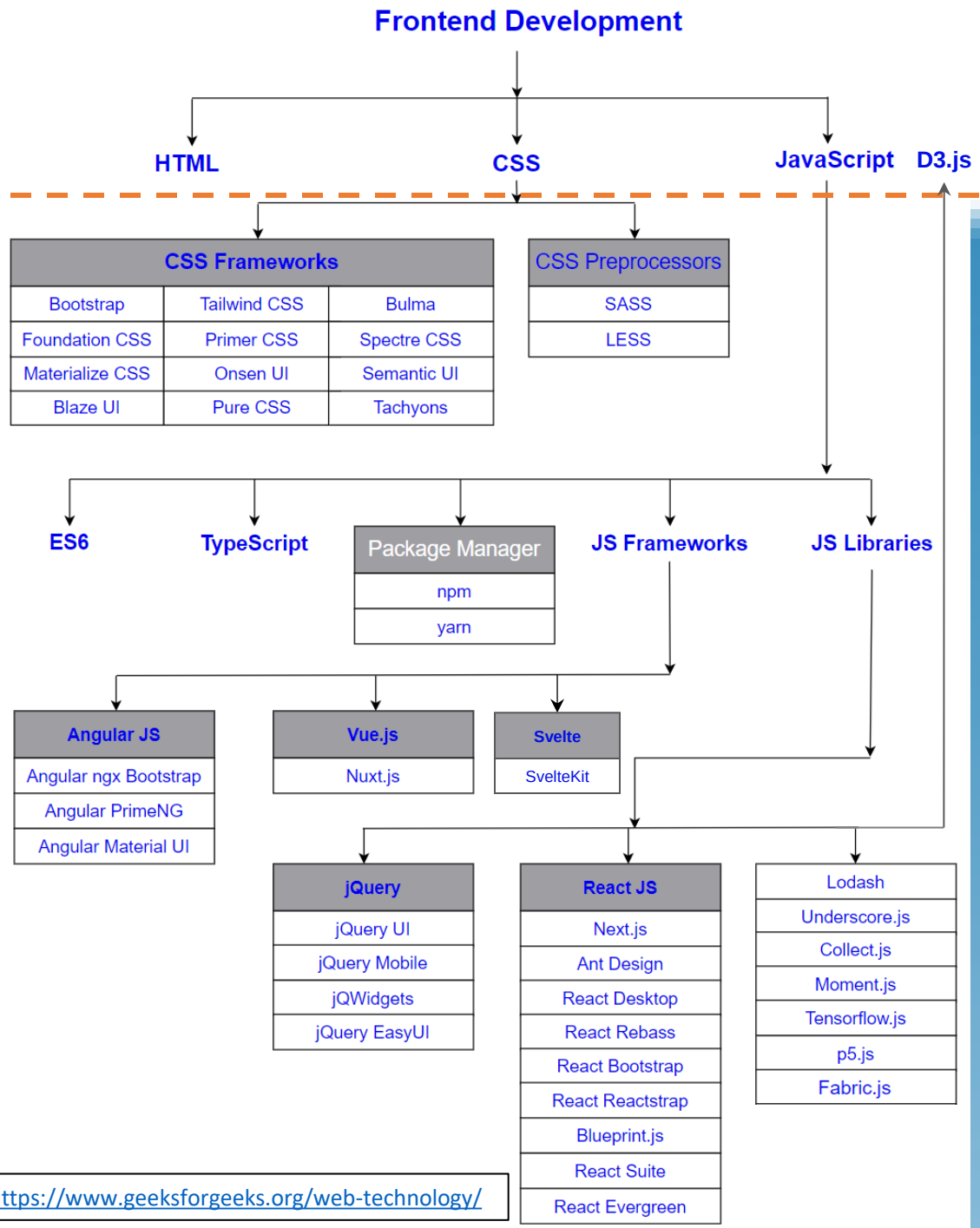


https://html-css-js.com/

What we'll mostly use in this course:

What else exists:

Web Technologies "Iceberg"



https://www.geeksforgeeks.org/web-technology/

…and we'll also use **SVG**

- Stands for **S**calable **V**ector **G**raphics
- SVGs are scalable to different display resolutions and color spaces.
- Uses the power of transformations, coordinate systems, units, and vector-based shapes.

An SVG Circle

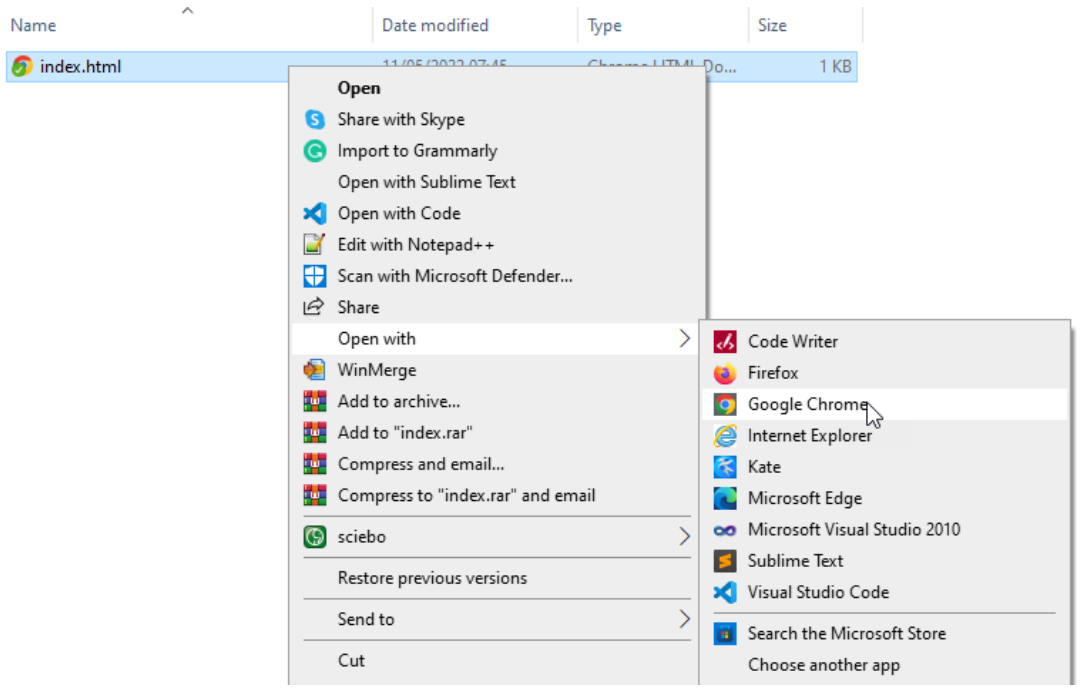
Try viewing it at different zoom levels and notice how the border scales.

Q: What if you screenshot this and then check it at different zoom levels?

https://www.w3schools.com/html/

Programming Setup

- No Web Server
  - Cannot link local files (e.g., .json)



Programming Setup

- No Web Server
  - Cannot link local files (e.g., .json)
- Local Web Server
  - Python Simple HTTP Server
  - python v3: python –m http.server
  - python v2: python -m SimpleHTTPServer

File Edit Selection Find View Goto Tools Project Preferences Help

index.html

```html
<!DOCTYPE html>
<html>
<head>
    <title>Introduction to D3.js</title>
</head>
<body>
    <h1>Welcome to the introduction of D3.js</h1>

</body>
</html>
```

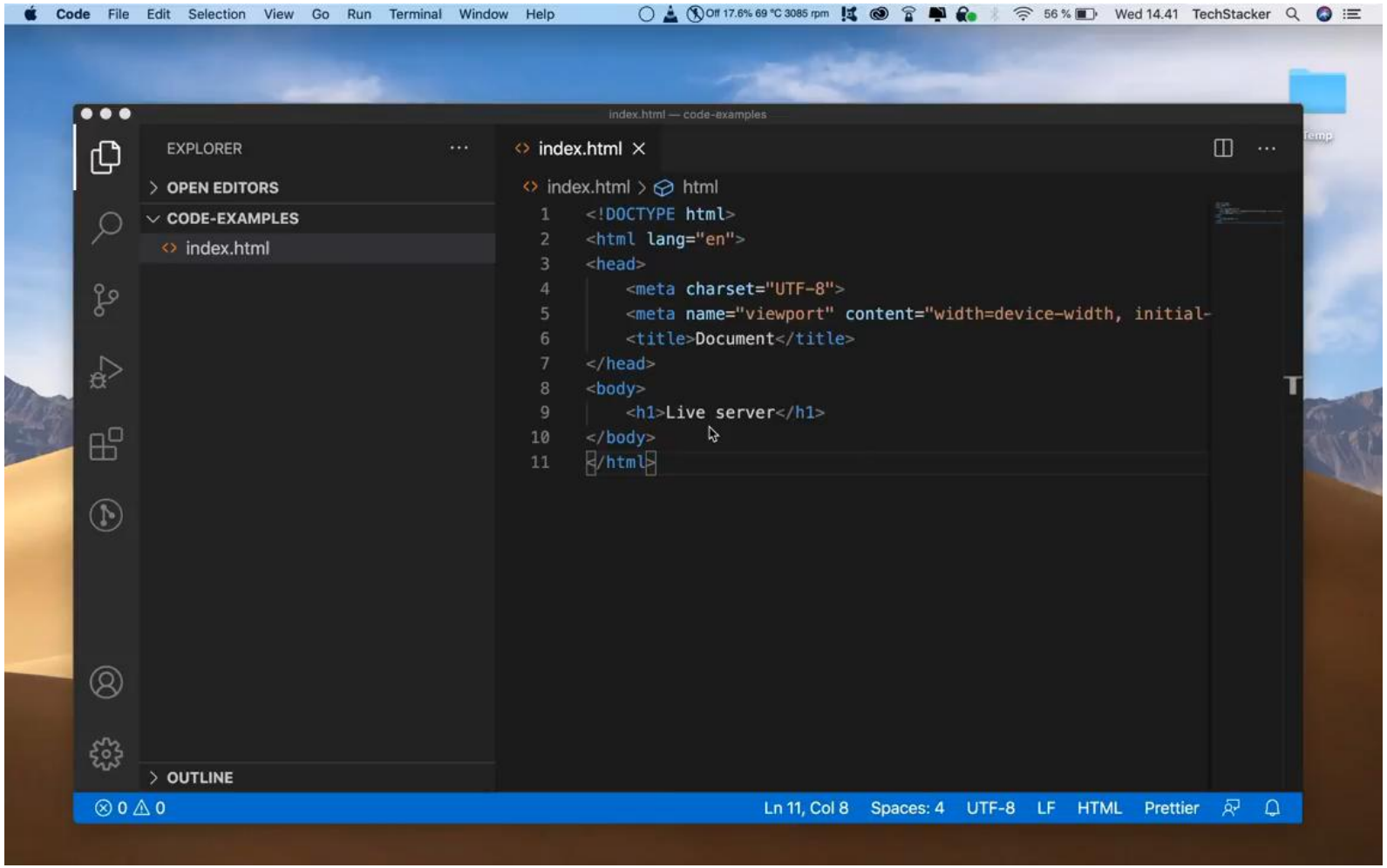
```
(base) C:\Users\Shivam>python -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

localhost:8000

Welcome to the introduction of D3.js

Programming Setup

- Local Web Server in IDE
  - e.g., Live Server in Visual Studio Code



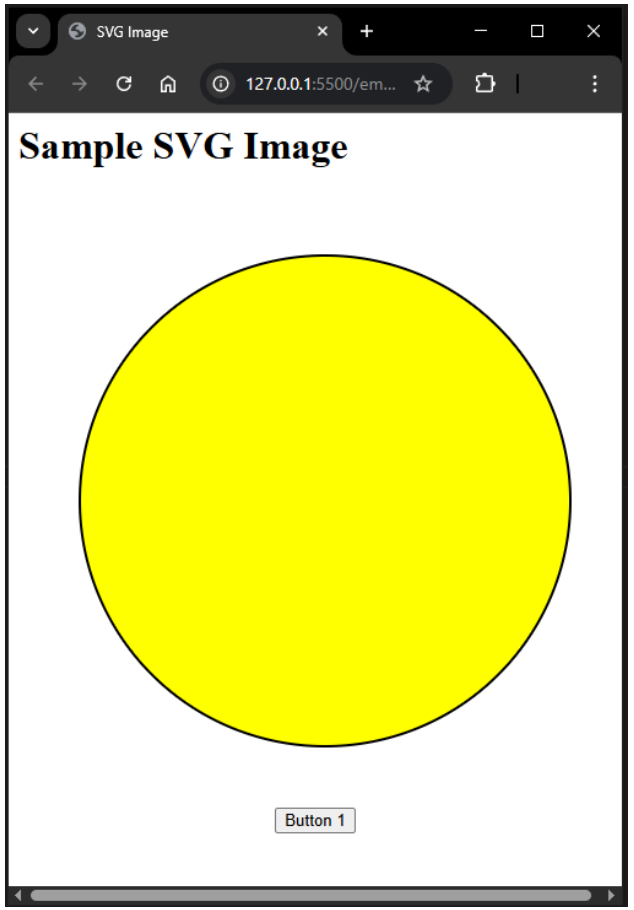
https://www.youtube.com/watch?v=_wue59ldqMg

Sample HTML Code:

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>SVG Image</title>
</head>

<body>

    <!-- Add a header for the HTML -->
    <h1>Sample SVG Image</h1>

    <!-- div defines a section (division) in the HTML document -->
    <div style="text-align: center;">

        <svg width="500" height="500" id="emoji_svg">
            <circle cx="250" cy="250" r="200" stroke="black" stroke-width="2" fill="yellow" />
        </svg>

        <!-- br is for adding a new line -->
        <br>
        <button type="button" id="button1">Button 1</button>
    </div>

</body>

</html>
```

emoji1.html

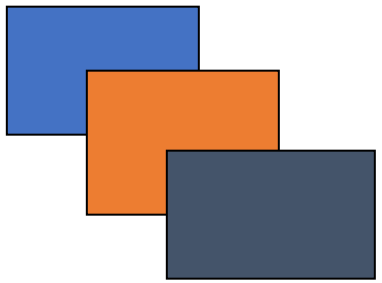
Output (browser):



Rendering SVG

- SVG content is drawn on an infinite 2D plane
- Drawing is done w.r.t. a finite region: **Viewport**
- Size of the **viewport** is defined by height and width in the <svg> element

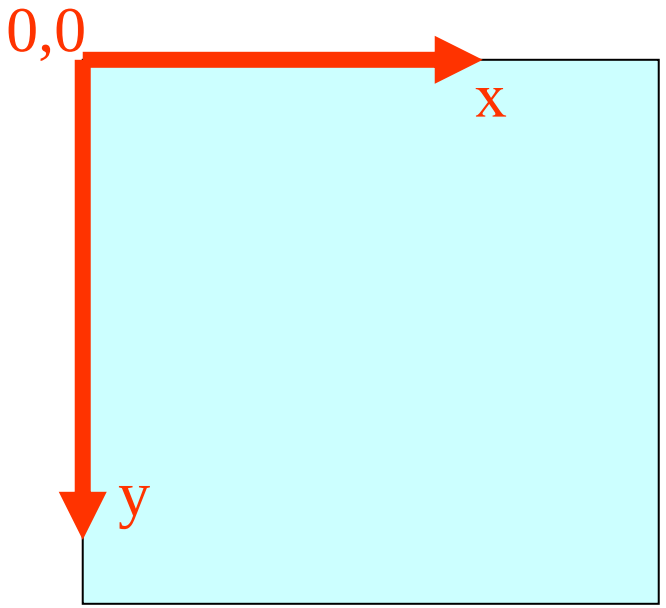
- Rendering order: **first comes, first painted**

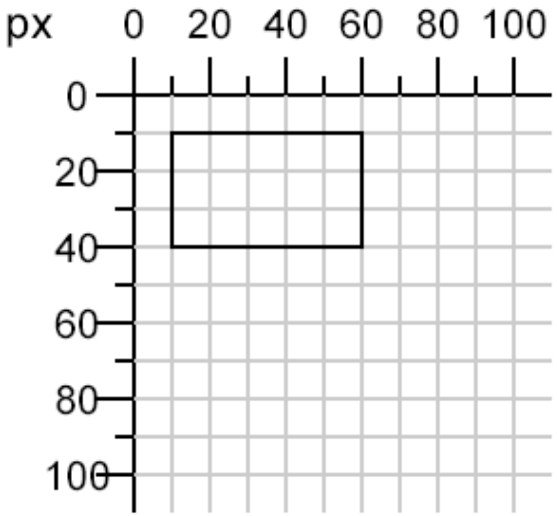


SVG Coordinate System

- SVG documents have their origin at the top left
- Supported units of measure:

pixels	**px** (default)
percent	%
millimeters	mm
centimeters	cm
inches	in
ems	em
x height	ex
points	pt
picas	pc



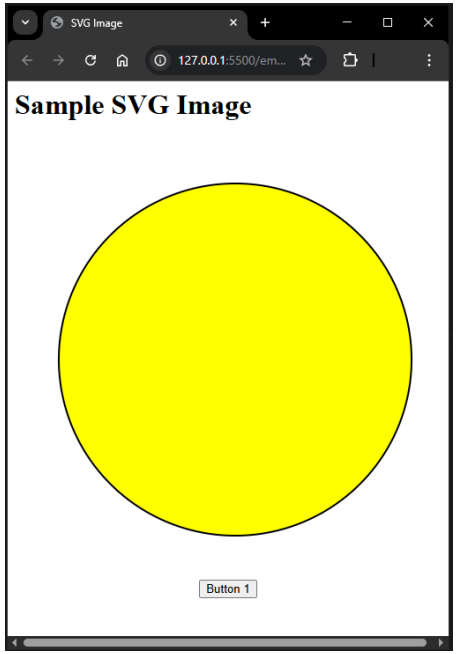



```
<svg width="200" height="200">
   <rect x="10" y="10"
         width="50"
         height="30"
          stroke="black"
         fill="none"/>
</svg>
```

Integrating SVG with HTML

```
<svg width="500" height="500" id="emoji_svg">
        <circle cx="250" cy="250" r="200" stroke="black" stroke-width="2" fill="yellow" />
</svg>
```

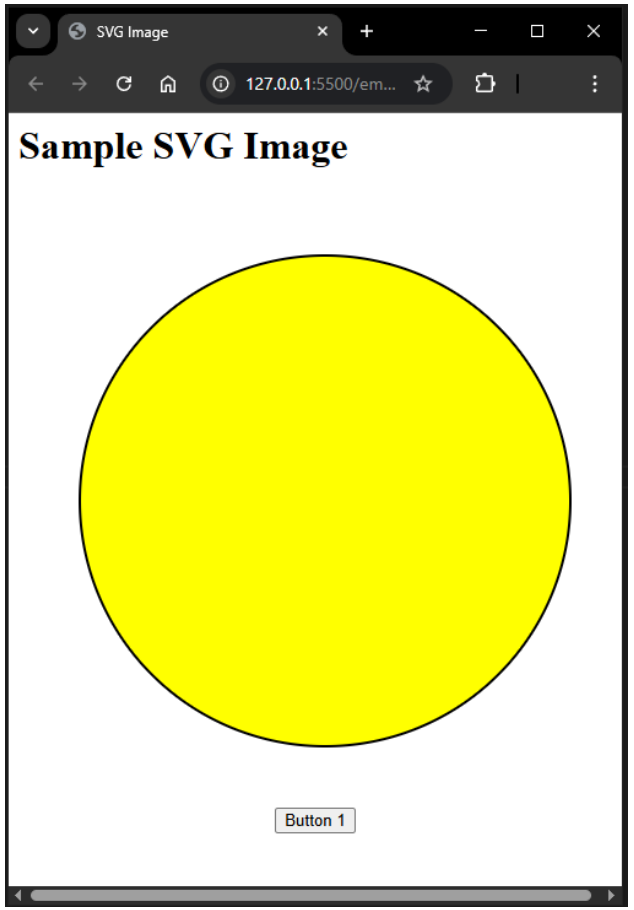
- **<svg>** element indicates the start of SVG image.
- <svg> element's **width** and **height** attributes defines the height and width of the SVG image.
- **cx** and **cy** attribute represents center of the circle. Default value is (0,0). **r** attribute represents radius of circle.
- Other attributes **stroke** and **stroke-width** controls the outlining of the circle.
- **fill** attribute defines the fill color of the circle.
- Closing **</svg>** tag indicates the end of SVG image.



Sample HTML Code:

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>SVG Image</title>
</head>

<body>

    <!-- Add a header for the HTML -->
    <h1>Sample SVG Image</h1>

    <!-- div defines a section (division) in the HTML document -->
    <div style="text-align: center;">

        <svg width="500" height="500" id="emoji_svg">
            <circle cx="250" cy="250" r="200" stroke="black" stroke-width="2" fill="yellow" />
        </svg>

        <!-- br is for adding a new line -->
        <br>
        <button type="button" id="button1">Button 1</button>
    </div>

</body>

</html>
```

Output (browser):

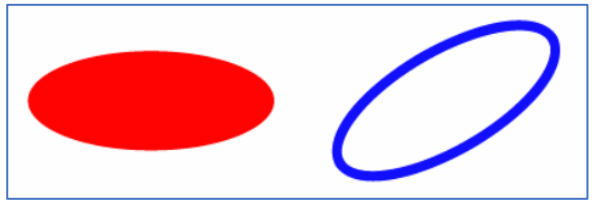


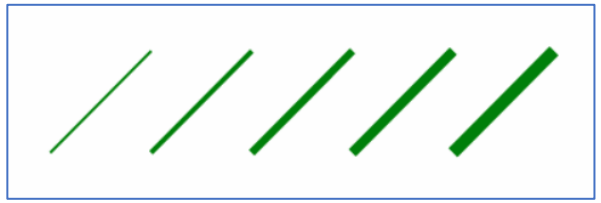
Activity:

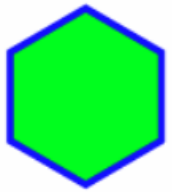
- Try running the **emoji1.html** (from the ZIP file in VC).
- Find out what happens when **cx**<0 or **cy**<0? Or what if you give larger values of **r**? Or, **r** as a percentage(r="20%")?
- Try *inspecting* elements.

Basic Shapes

- All basic shapes in SVG are represented as elements, with attributes corresponding to the shape.
- **<rect>** , **<circle>**, **<ellipse>**, **<line>**
- **<polyline>** - creates straight lines between multiple points.
- **<polygon>** - defines a closed shape consisting of a set of connected straight line segments.
- Each shape can be stroked, filled and transformed.

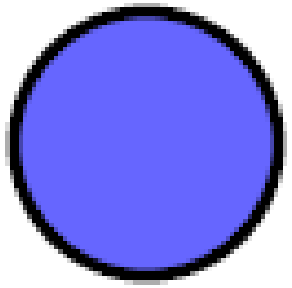






Basic Shapes (2)

```
<svg width="200" height="200">
   <circle cx="60" cy="60" r="15"
         stroke="black"
         stroke-width="1.5"
         fill="blue"
         fill-opacity="0.6" />
</svg>
```



```
<svg width="120" height="120">
   <polygon
         points="48,16 16,96 96,48 0,48 80,96"
         stroke="black"
         stroke-width="1"
         fill="none" />
</svg>
```

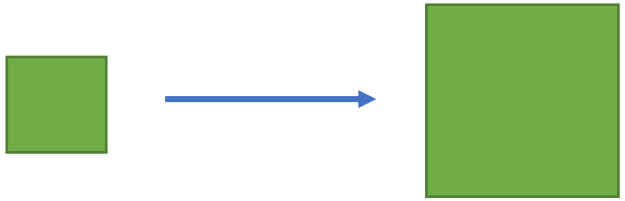
```
<svg width="120" height="120">
         <polyline points="5,20 20,20 25,10
                           35,30 45,10 55,30
                           65,10 75,30 80,20 95,20"
         stroke="black"
         stroke-width="3"
         fill="none"/>
</svg>
```

```
<svg width="30" height="10">
   <g fill="red">
      <rect x="0" y="0" width="10" height="10" />
      <rect x="20" y="0" width="10" height="10" />
   </g>
</svg>
```

Basic SVG Transformations

<g> is for defining a group of SVG elements.

- **Scaling**: scale(a)

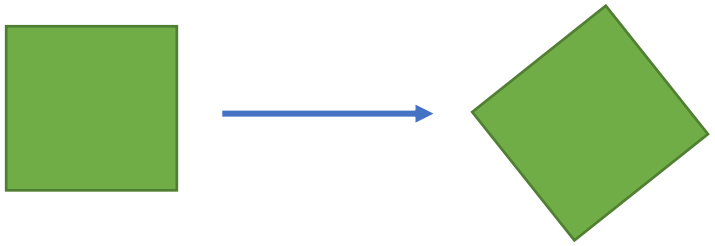


```
<svg width="100" height="100">
    <g transform="scale(2)">
        <rect width="50" height="50" />
    </g>
</svg>
```

- **Translation**: translate(x,y)

```
<svg width="40" height="50">
    <rect x="0" y="0" width="10" height="10"
transform="translate(30,0)" />
</svg>
```

- **Rotation**: rotate(alpha)



```
<svg width="31" height="31">
    <rect x="12" y="-10" width="20" height="20"
transform="rotate(45)" />
</svg>
```

Activity: Modify the SVG in **emoji1.html** (from the ZIP file in VC) to get the above shown emoji.

For reference:

SVG Circle - <circle>

The `<circle>` element is used to create a circle.

The `<circle>` element has three basic attributes to position and set the size of the circle:

Attribute	Description
r	Required. The radius of the circle
cx	The x-axis center of the circle. Default is 0
cy	The y-axis center of the circle. Default is 0

https://www.w3schools.com/graphics/svg_circle.asp

SVG Line - <line>

The `<line>` element is used to create a line.

The `<line>` element creates a line between the start position (x1,y1) and the end position (x2,y2).

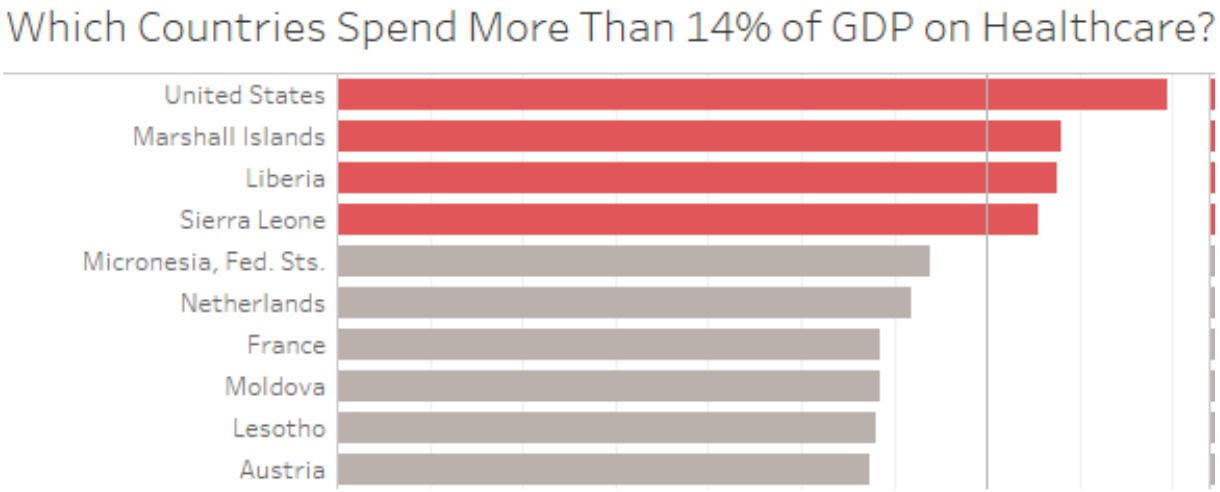
The `<line>` element has four basic attributes to position and set the length of the line:

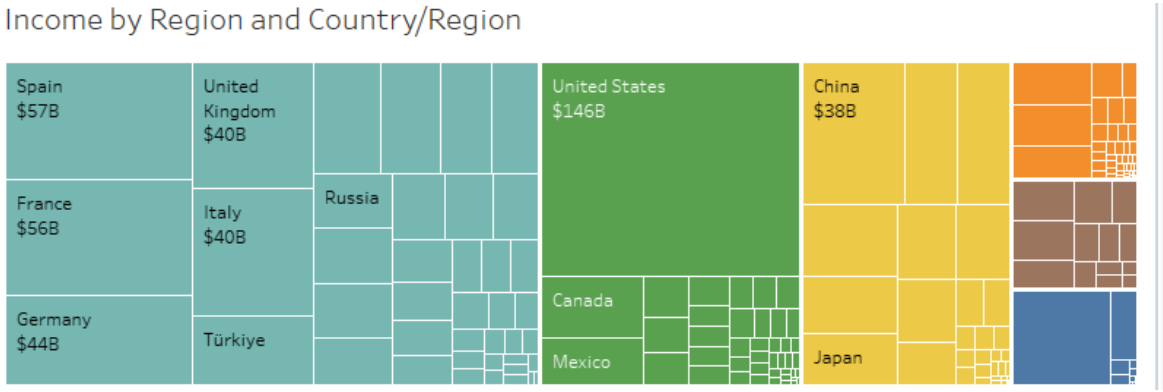
Attribute	Description
x1	The start of the line on the x-axis
y1	The start of the line on the y-axis
x2	The end of the line on the x-axis
y2	The end of the line on the y-axis

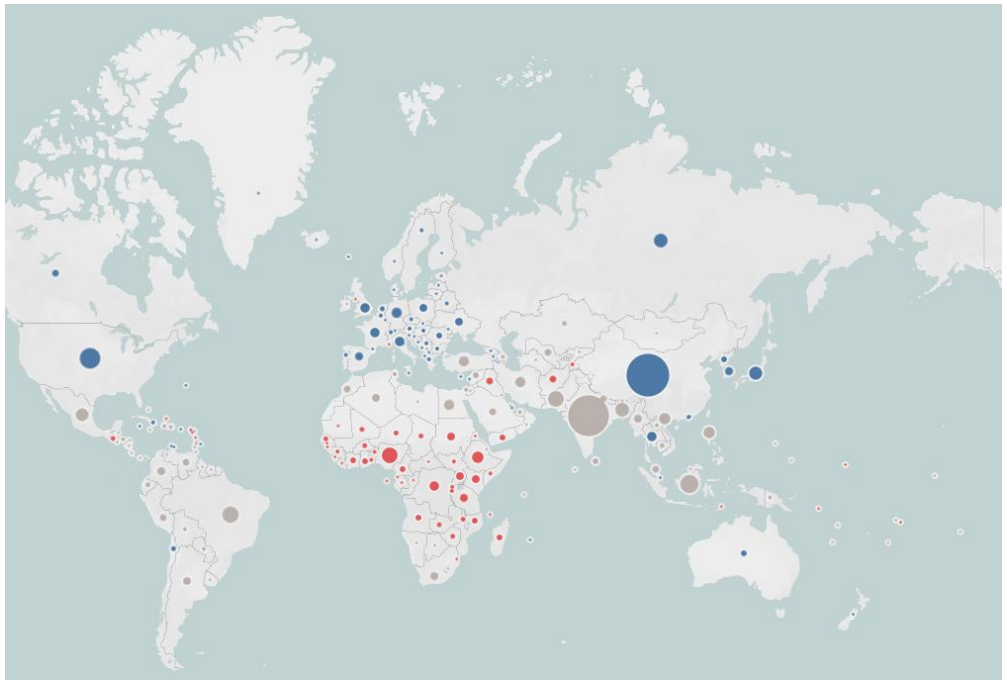
https://www.w3schools.com/graphics/svg_line.asp

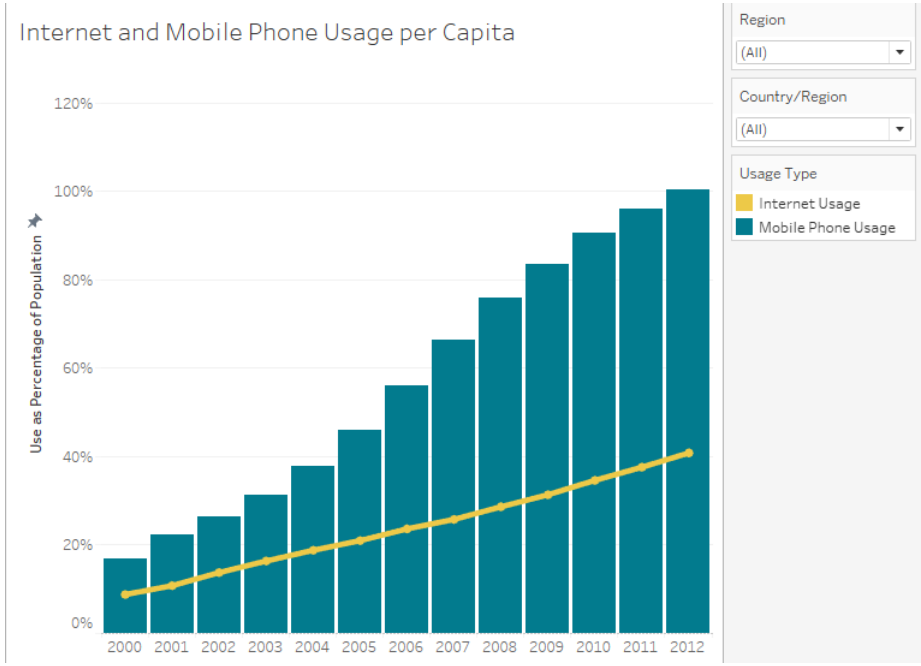
Output (browser):

Activity: Recognize the SVG Elements in Visualizations

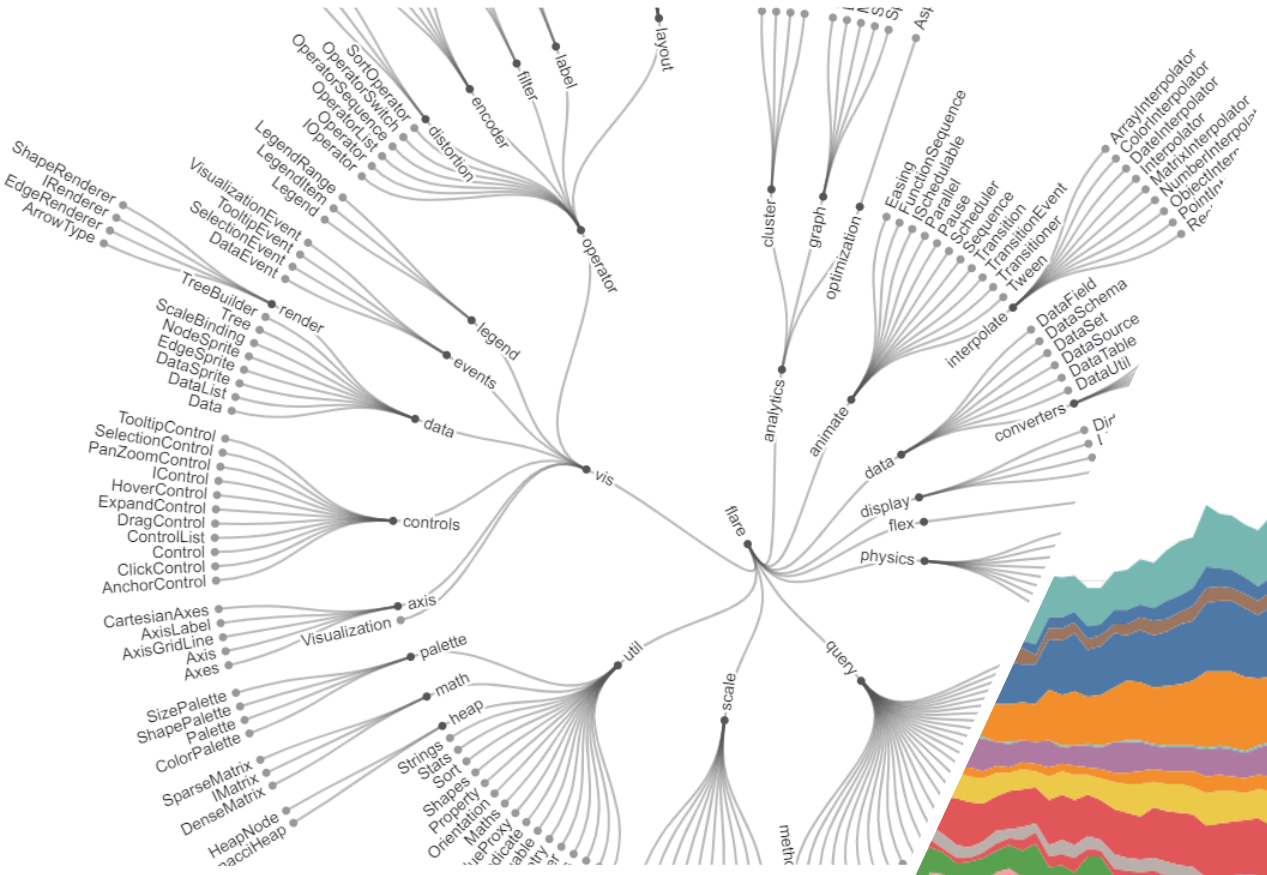


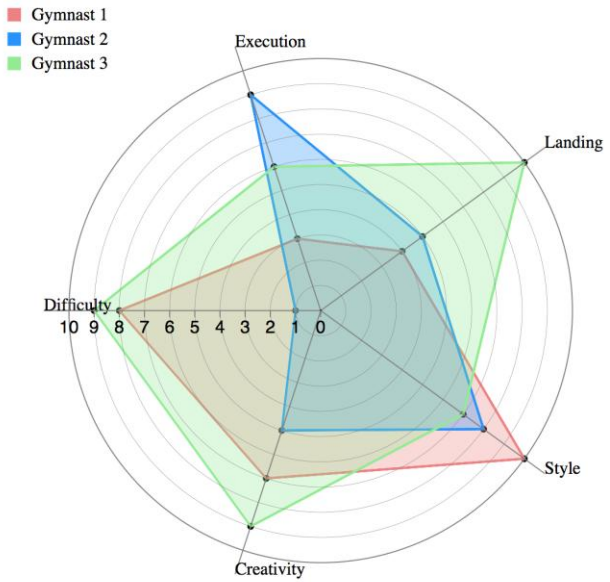


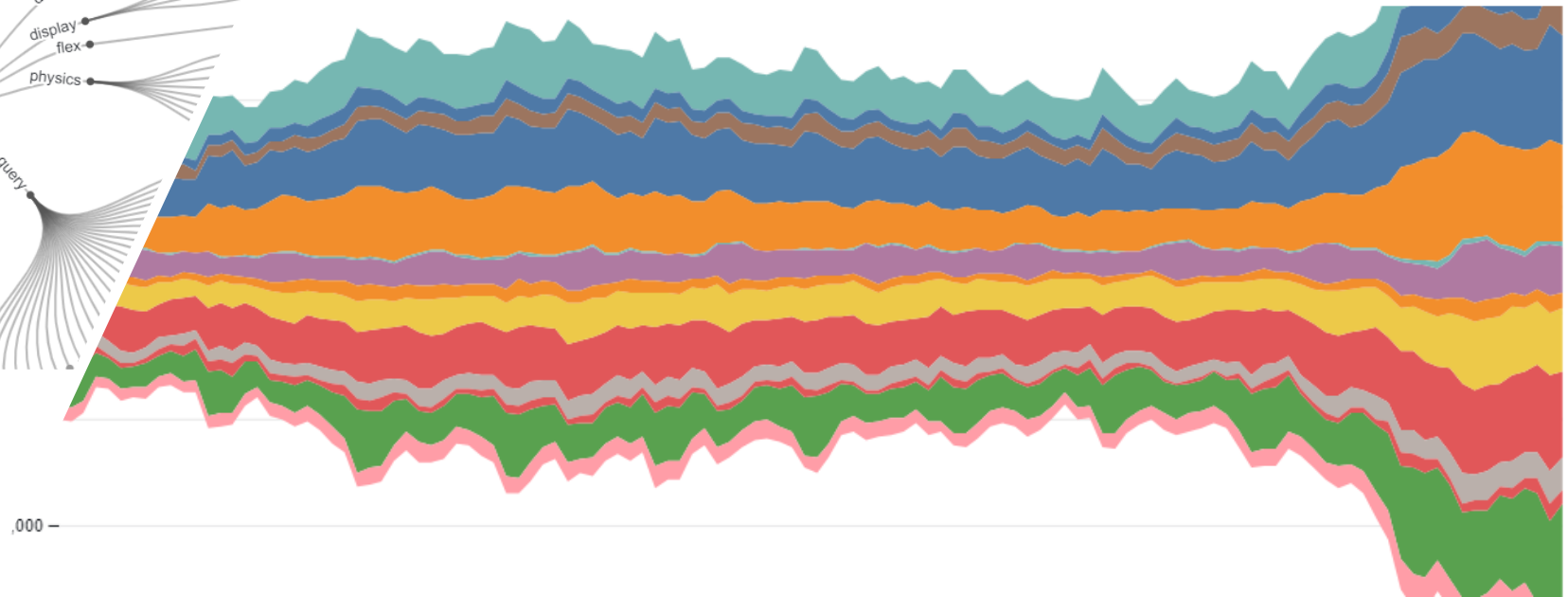




Activity: Recognize the SVG Elements in Visualizations







SVG <path>

- Create simple or complex shapes

Attribute	Description
d	Required. A set of commands which define the path. The following commands are available for path data: • M = moveto (move from one point to another point) • L = lineto (create a line) • H = horizontal lineto (create a horizontal line) • V = vertical lineto (create a vertical line) • C = curveto (create a curve) • S = smooth curveto (create a smooth curve) • Q = quadratic Bézier curve (create a quadratic Bézier curve) • T = smooth quadratic Bézier curveto (create a smooth quadratic Bézier curve) • A = elliptical Arc (create a elliptical arc) • Z = closepath (close the path) **Note:** All of the commands above can also be expressed in lower case. Upper case means absolutely positioned, lower case means relatively positioned.

https://www.w3schools.com/graphics/svg_path.asp

https://www.w3schools.com/css/

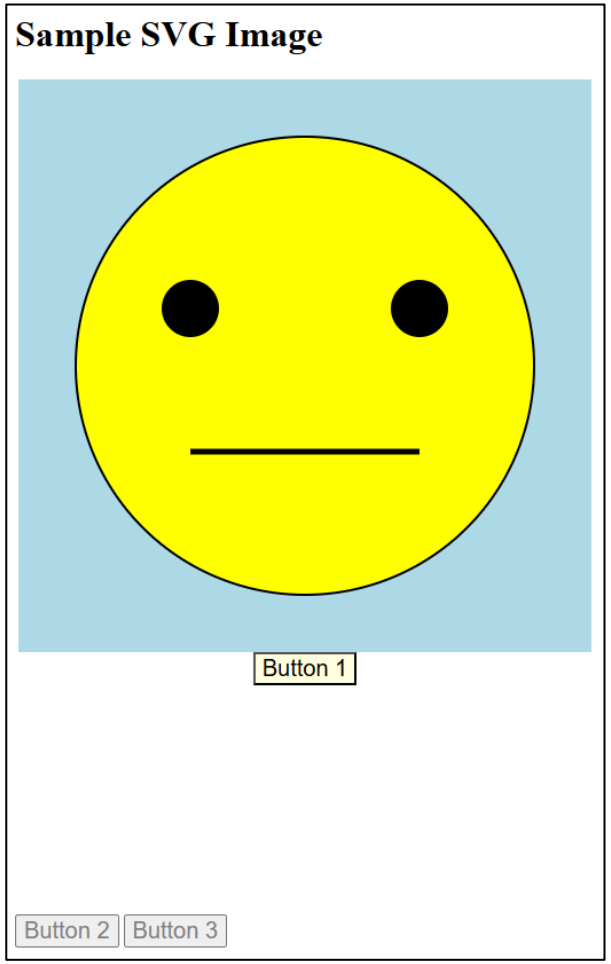
HTML Code containing the CSS (<style>…</style>):

```
<style>
    button {
        font-size: 20px;
    }

    .uselessbuttons {
        margin-top: 50px;
        color: gray;
    }

    #button1 {
        background-color: lightyellow;
    }

    #button1:hover {
        background-color: yellow;
    }

    svg {
        background-color: lightblue;
    }
</style>

<body>

… rest of the code
```

Buttons in the HTML

```
<div>
   <!-- some code --!>
   <button type="button" id="button1">Button 1</button>
</div>


   <button type="button" class="uselessbuttons">Button 2</button>
   <button type="button" class="uselessbuttons">Button 3</button>



… rest of the code
```

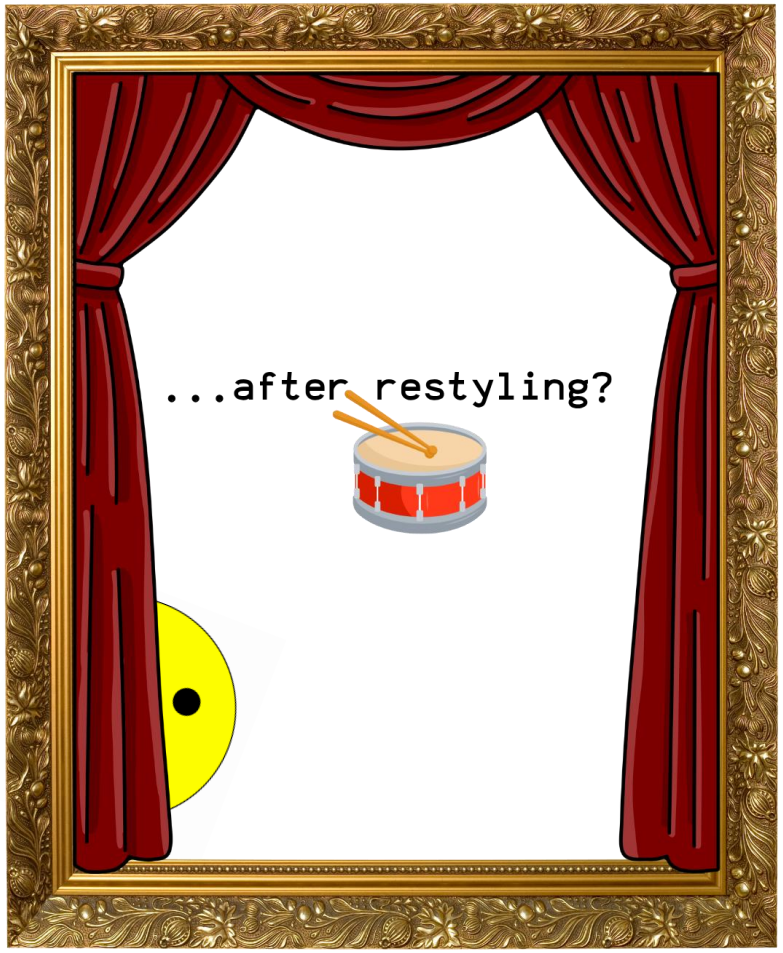
Output (browser):



Note: Code available in emoji2.html

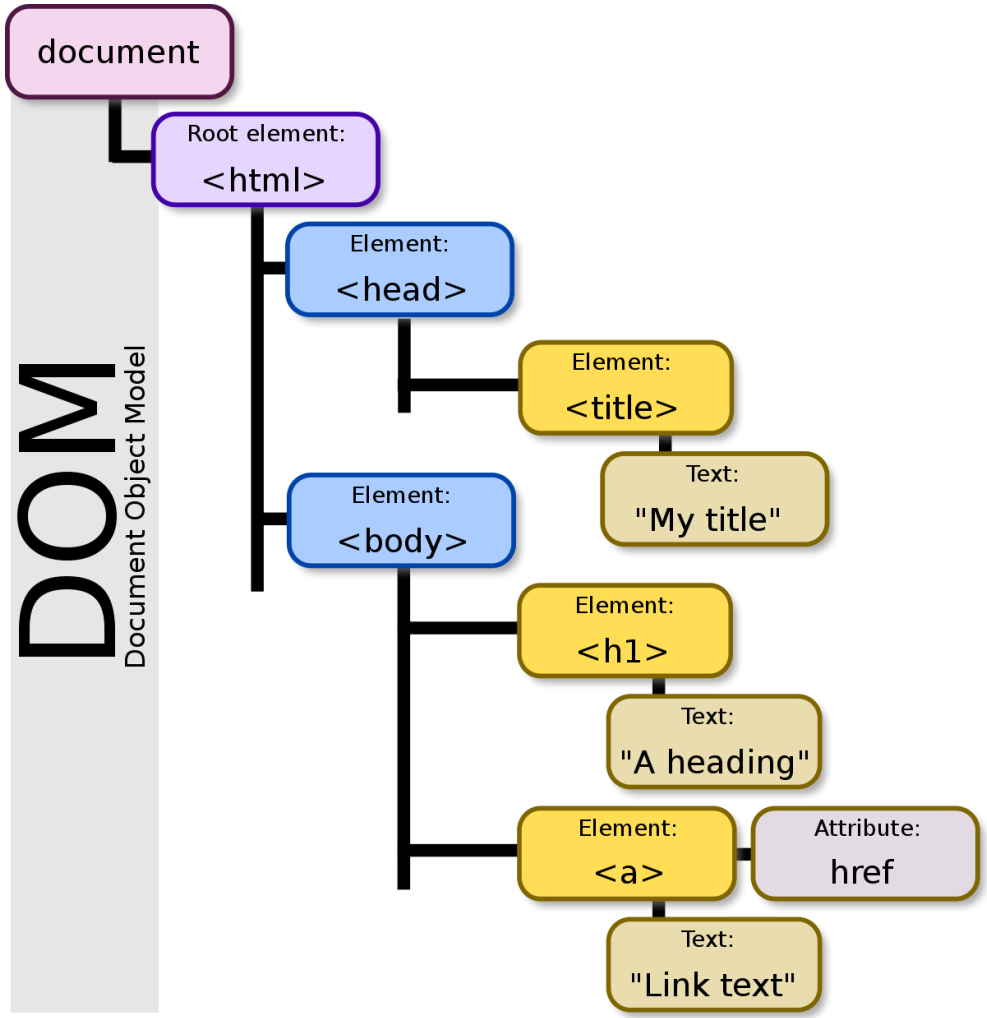
Activity

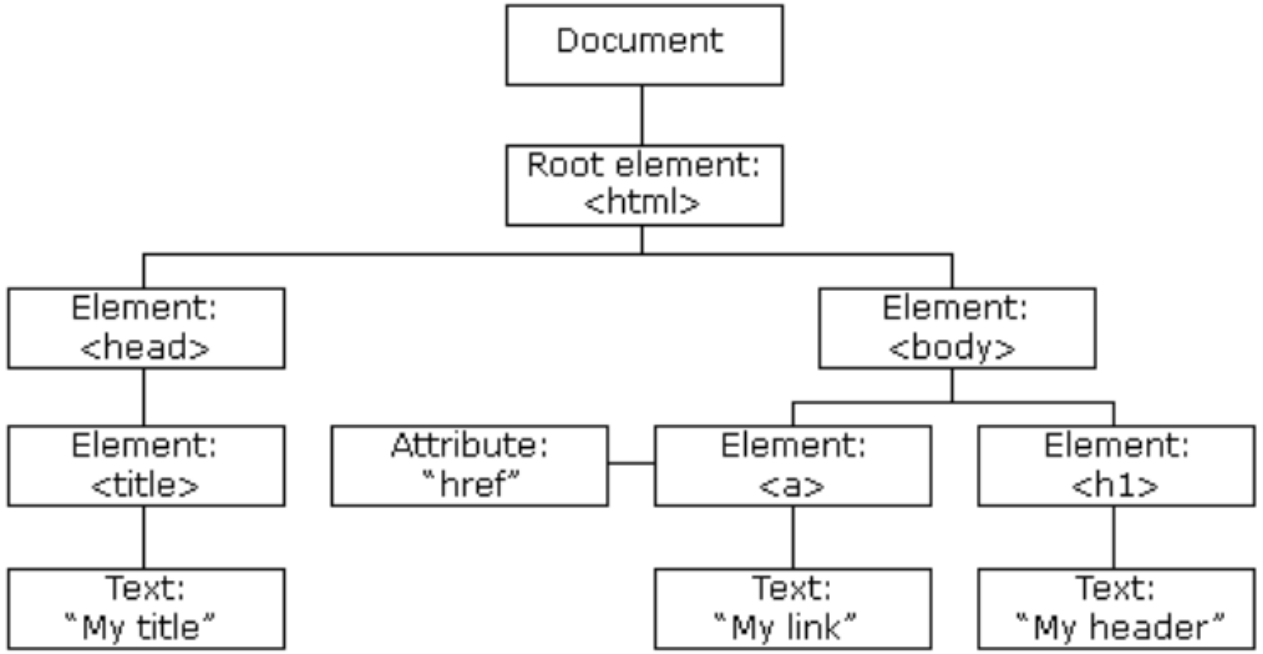
- Use CSS to style the smiley eyes in a more unique way.



https://slides.com/concise/js/fullscreen/#/

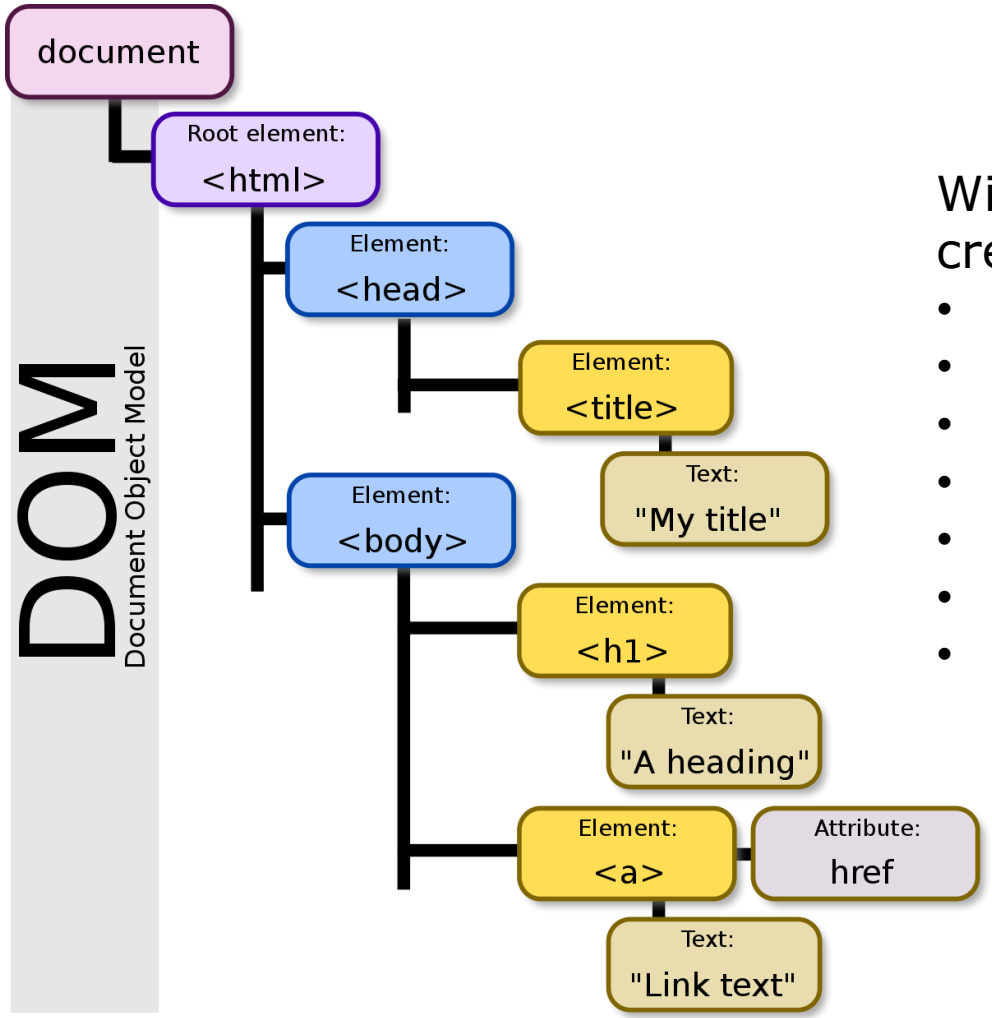
Document Object Model (DOM)





https://www.w3schools.com/js/js_htmldom.asp

JS + DOM



With the DOM, JavaScript gets all the power it needs to create dynamic HTML:

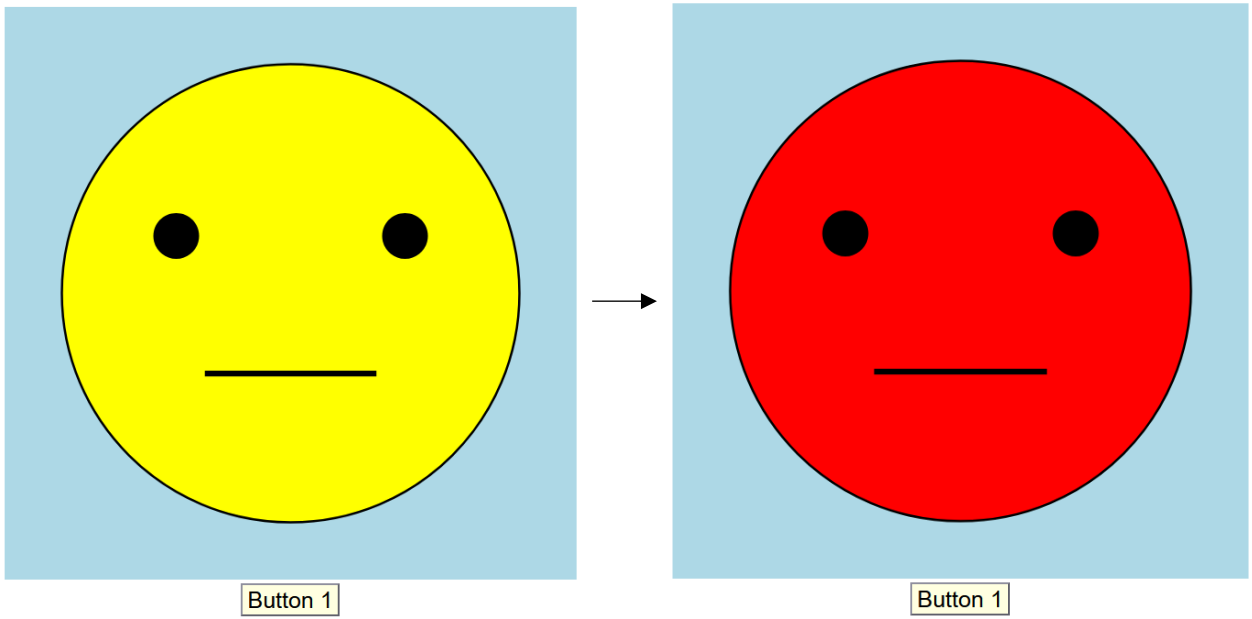
- JS can change all the HTML elements in the page
- JS can change all the HTML attributes in the page
- JS can change all the CSS styles in the page
- JS can remove existing HTML elements and attributes
- JS can add new HTML elements and attributes
- JS can react to all existing HTML events in the page
- JS can create new HTML events in the page

https://www.w3schools.com/js/js_htmldom.asp

HTML Code containing the JS (<script>…</script>):

```
… rest of the code


<script>

function toggleSVGColor() {
    console.log("Button 1 clicked!");

    const face = document.getElementById("face");

    let currentColor = face.getAttribute("fill");

    if (currentColor === "yellow") {
        face.setAttribute("fill", "red");
    } else {
        face.setAttribute("fill", "yellow");
    }
}

    // Get the button element
    const button1 = document.getElementById('button1');
    // When the button is clicked...
    button1.addEventListener('click', toggleSVGColor);

    //TODO: Make the left eye larger when hovered

</script>
```

Output (browser):

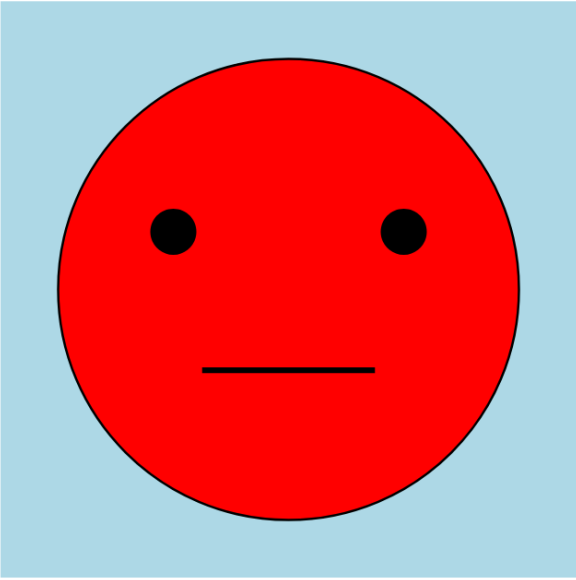



```
<circle id="face" cx="250" cy="250" r="200"
stroke="black" stroke-width="2" fill="yellow" />
```

Note: Code available in emoji2.html

Activity

```
//TODO: Make the left eye larger when hovered
```



Hints:

```
"mouseover"
                addEventListener
id=

.setAttribute
```

Note: Base for code available in emoji2.html